

Chapter 4

Computer Arithmetic



Computer Arithmetic

- Integer Arithmetic
- Floating point Arithmetic



Integer Representation

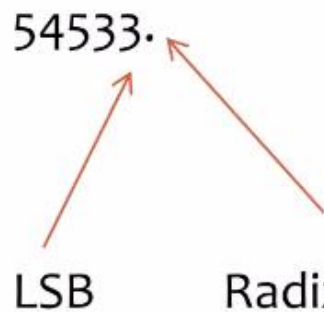
- ❑ Integers are whole numbers or fixed point numbers. The radix point is fixed after the LSB.

Example:

3774_{10} is a fixed point number (integer)

2949.94_{10} is a floating point number (real number)

54533.



LSB Radix Point Fixed

The diagram shows the number 54533. with a red arrow pointing from the text 'LSB' to the last digit '3' and another red arrow pointing from the text 'Radix Point Fixed' to the decimal point.

4573.55



Radix Point Moved from LSB

The diagram shows the number 4573.55 with a red arrow pointing from the text 'Radix Point Moved from LSB' to the decimal point.

Integer Representation

- ❑ In a computer integers and floating point numbers are treated differently and processed separately. Floating point numbers are processed in a separate processor called floating point processor.
- ❑ Computers use fixed number of bits to represent integers. Commonly use bit lengths are 8bit, 16bit, 32bit and 64bit.
- ❑ There are two types of integers
 - ❑ Unsigned integers: can represent zero and positive numbers only
 - ❑ Signed integers: can represent zero, positive and negative numbers.
- ❑ There are three schemas (ways) to represent signed integers.
 - ❑ Sign Magnitude representation
 - ❑ 1's complement representation
 - ❑ 2's complement representation

Integer Representation

- We learned about two different types of Integers;
 - Unsigned Integers – that can represent only zero and positive numbers.
The value of an unsigned integer interpreted as the “magnitude of underlying binary pattern or the normal way we convert binary to decimal.

Example: 8bit binary number 0100 0010B (B = Binary)
 $0100\ 0010B = 2 \times 2^1 + 2 \times 2^6 = 4 + 64 = 68D$ (D = Decimal)

8bit = 0000 0000

16bit = 0000 0000 0000 0000

32bit = 0000 0000 0000 0000 0000 0000 0000 0000

64bit = 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000

All above numbers represent zero in different bit systems. The bit length of each system is different.

n-bit Unsigned Integers

- ❑ An n-bit pattern can represent 2^n distinct integers. This range from 0 to $(2^n)-1$

n-bit	Minimum	$2^n - 1$	Maximum
8bit	0	$2^8 - 1$	255
16bit	0	$2^{16} - 1$	65,535
32bit	0	$2^{32} - 1$	4,294,967,295
64bit	0	$2^{64} - 1$	18,446,744,073,709,551,615

When you are programming you need to select the correct and appropriate bit size to store integers. If you want to store small numbers less than 255, you can select 8bit number, selecting 32bit length to store small numbers is a waste of resources.

Signed Integers

- ❑ Signed integers can represent zero, positive and negative numbers.
- ❑ The MSB (Most Significant Bit) – Normally the first bit from left, represent the sign of the integer. Zero (0) to represent positive (+) sign and One (1) to represent negative (-) sign.
- ❑ The magnitude of the integer represent differently in different schemas.
- ❑ There are three schemas to represent signed integers.

1. Sign Magnitude Representation
2. 1's Complement Representation
3. 2's Complement Representation

Example of Signed Integers:

+2344 is a positive integer

-23434 is a negative integer

n-bit Sign Magnitude Representation

- ❑ Most Significant Bit (MSB) is the sign bit. 0 representing positive and 1 representing negative signs.
- ❑ The remaining (n-1) bits represent the magnitude / absolute value of the integer interpreted according to its binary pattern.

Examples:

n=8 (8bit number) and binary number is 01000001

Sign bit (MSB) is 0 → this is a positive number

Absolute value 1000001 → 65

01000001B = +65D

B = Binary, D = Decimal

n-bit Sign Magnitude Representation

Examples:

n=8 (8bit number) and binary number is 00000000

Sign bit (MSB) is 0 → this is a positive number

Absolute value 0000000 → 0

00000000B = +0D

n=8 (8bit number) and binary number is 10000000

Sign bit (MSB) is 1 → this is a negative number

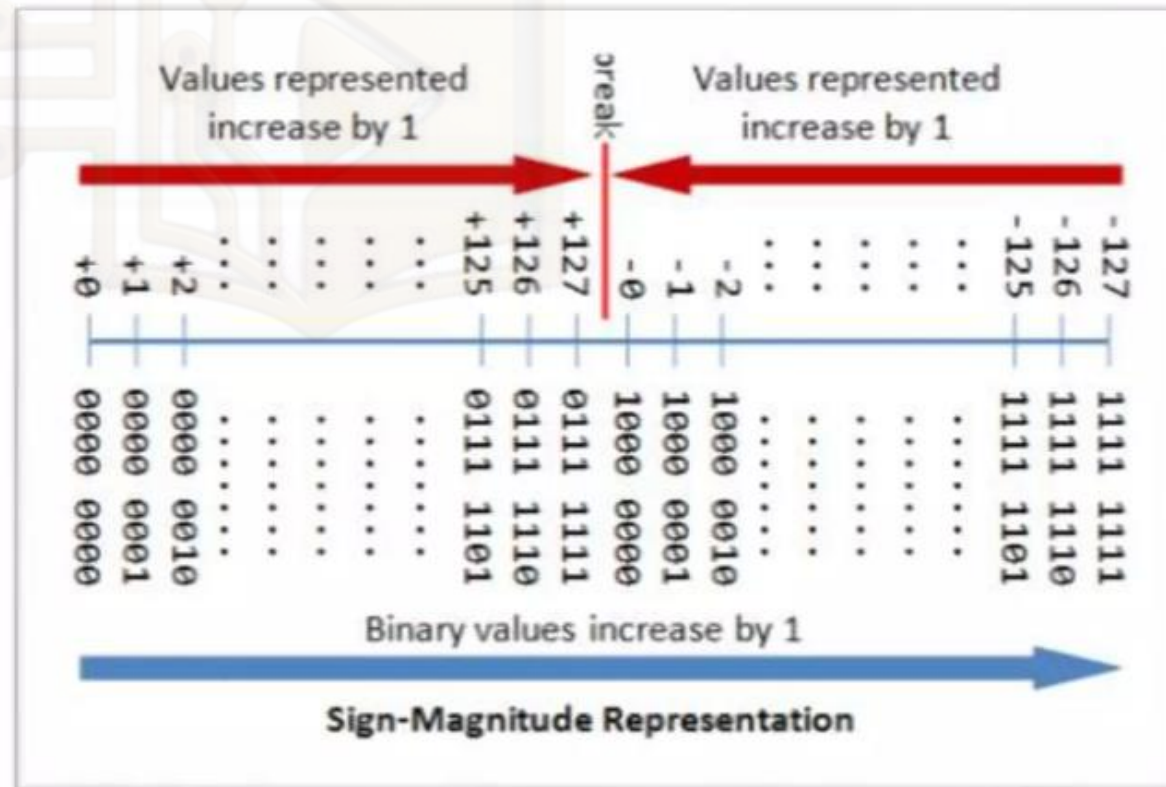
Absolute value 0000000 → 0

10000000B = -0D

n-bit Sign Magnitude Representation

Problems (Drawbacks / Limitations) of Sign Magnitude Representation:

- ❑ There are two representation for the number zero
- ❑ Even the number zero need to have a sign, since it is not necessary nor accurate and may leads to confusions.
- ❑ Positive and negative numbers need to process separately.



n-bit 1's Complement Representation

In this representation, the MSB is the sign bit 0 represent positive and 1 represent negative numbers. The remaining n-1 bits represent the magnitude (value) of the integer.

- ❑ In positive integers, the value of the integer is the binary pattern (value) of the n-1 bit magnitude. Same as Sign Magnitude Representation.
- ❑ For negative integers, the absolute value is the n-1 binary pattern value of the inverse magnitude (complement).

Example: 1 000 0001B (sign bit = 1, negative integer)

Complement of 000 0001 is 111 1110 = 126D

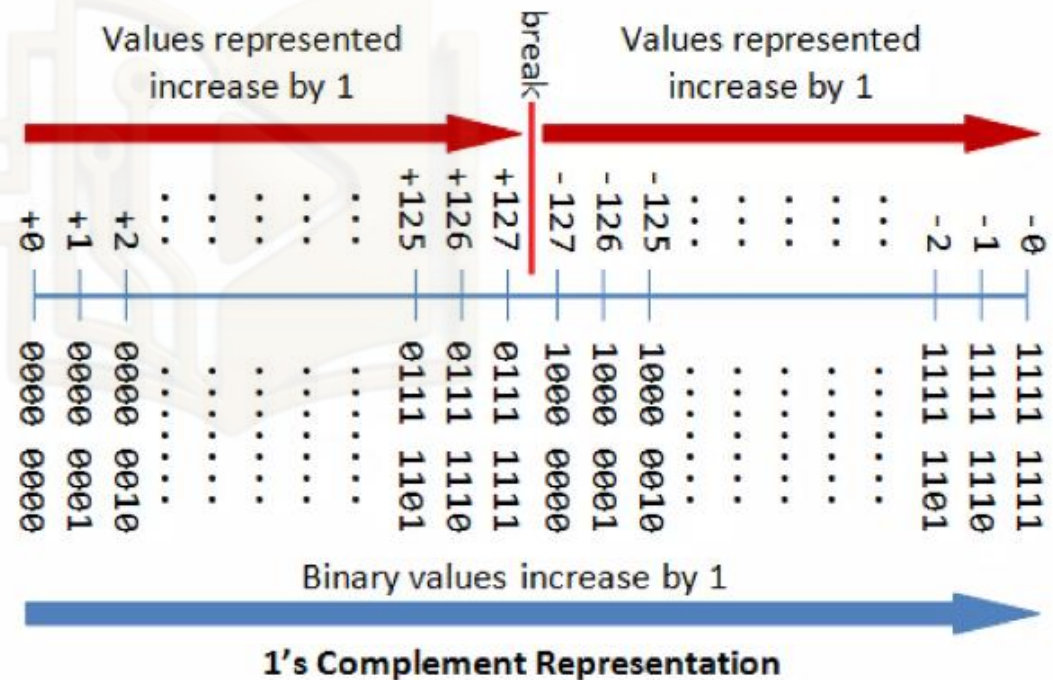
1 000 0001B = -126D

n-bit 1's Complement Representation

In the 1's Complement Representation, there are two different representation for zero;

0000 0000B and 1111 1111B
hence +0 and -0

Since the processing method of positive and negative integers are different, computer need to process the positive and negative numbers separately.



n-bit 2's Complement Representation

In 2's Complement Representation also the MSB is the sign bit 0 represent positive and 1 represent negative integers.

For positive integers, the value of the integer equal to the magnitude of n-1 binary pattern

For negative integers, the absolute value of the integer equal to the magnitude of the complement of n-1 binary pattern plus one.

Example: 1 000 0001B (MSB = 1 is a negative number)

Complement of 000 0001 = 111 1110

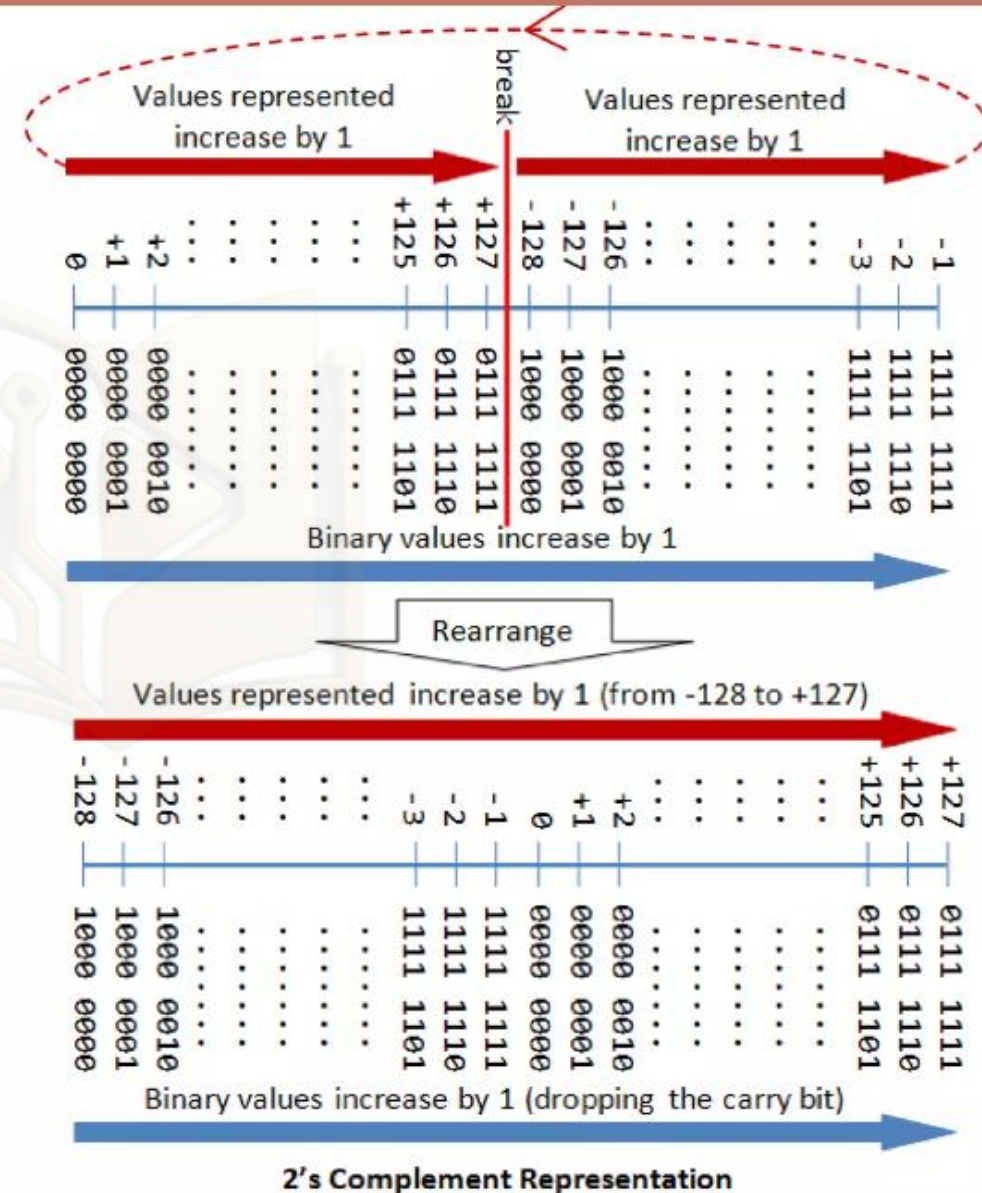
Let's add 1 to the complement = 111 1111 = 127

1 000 0001B = -127D

n-bit 2's Complement Representation

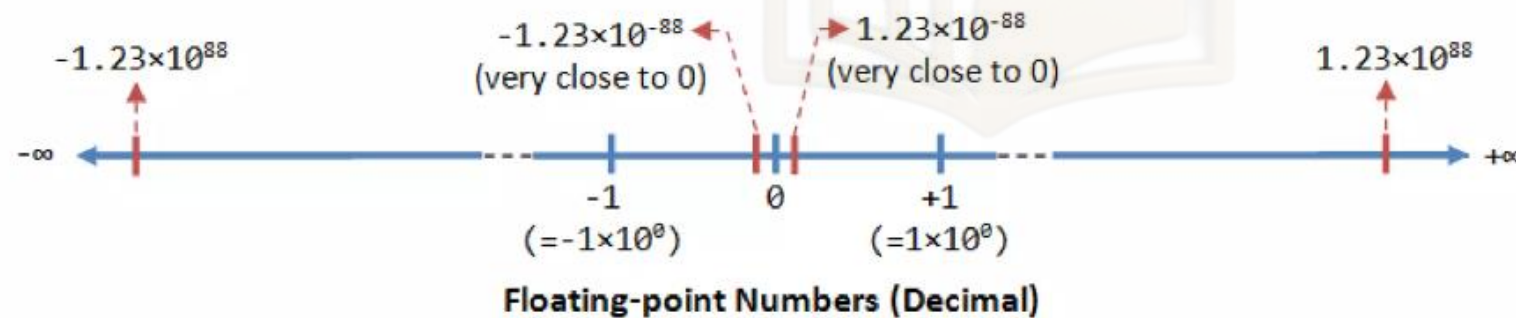
Advantages of 2's Complement Representation

- ❑ There is only one representation for the number 0
- ❑ In binary addition and subtraction, both positive and negative numbers can be treated together (process together)
- ❑ Subtraction can use the addition logic
- ❑ Computers use 2's Complement Representation for processing integers.



Floating Point Number Representation

A floating point number (also called as a real number) is a number with a decimal point. A floating point number can represent a very large positive or negative number as well as a very small negative or positive number. A real number has a range of -1.23×10^{88} to 1.25×10^{88} including zero.



Scientifically a floating point number represent as a number with a fraction and exponent of a certain radix. $F \times r^E$

Floating Point Number Representation

Same floating point number can represent in different ways. Consider 5.456D

We can represent this number in following ways:

$$54.532 \times 10^0$$

$$5.4532 \times 10^1$$

$$0.54532 \times 10^2 \text{ and so on....}$$

Since there are many ways to represent the same floating point number, we need to define a standard, so every floating point number should be in this standard. To make it standard, we can normalized the fraction part. In normalized form, there is only single none zero digit represent before the radix point. In above example 5.4532×10^1 is the normalized form of 54.532 number.

Floating Point Number Representation

In computers, there is an issue of “Loss of Precision” in storing floating point numbers. An n -bit system can only store 2^n distinct finite numbers. Even within a small range $0.00 - 0.01$, there are infinite number of decimal numbers. But the number of bits in a computer system is limited, only an approximate number is used, resulting in loss of accuracy.

Processing of a real number (floating number arithmetic) is very much less efficient and slower than processing of integers. In computers, real numbers use fraction (F) and exponent (E) with radix (r) of 2. Both F and E can be negative or positive. In modern computing, the IEEE 754 standard is used to represent floating point numbers. There are two representation schemes

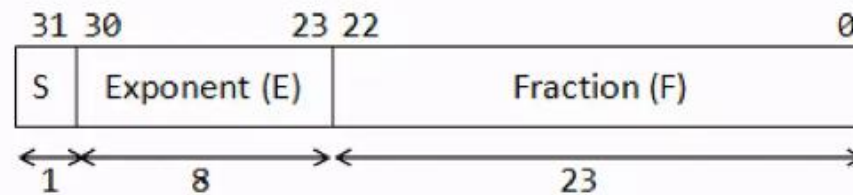
32 bit single precision and
64 bit double precision

IEEE-754 32-bit Single-Precision Floating-Point Numbers

In 32-bit single precision floating point number representation, decimal number represent in following structure:

- The most significant bit is the sign bit. Zero for positive numbers and One for negative numbers.
- Next following 8bit represent the exponent (E)
- Remaining 23bit represent the fraction (F)

You may wonder why radix (r) is not represent here. Because radix is always 2 for binary numbers (in computers).

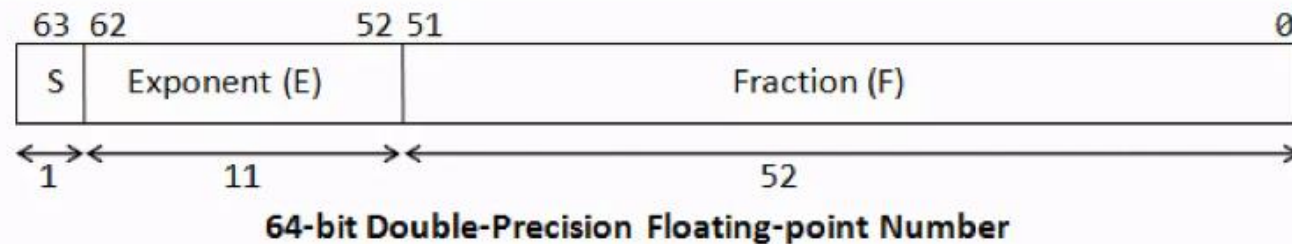


32-bit Single-Precision Floating-point Number

IEEE-754 64-bit Double-Precision Floating-Point Numbers

64-bit double precision floating point number representation is similar to 32-bit single precision floating point number representation. Only difference is assignment of bit range for exponent and fraction.

- The most significant bit is the sign bit. Zero for positive numbers and One for negative numbers.
- Next following 11-bit represent the exponent (E)
- Remaining 52-bit represent the fraction (F)



INTEGER ARITHMETIC

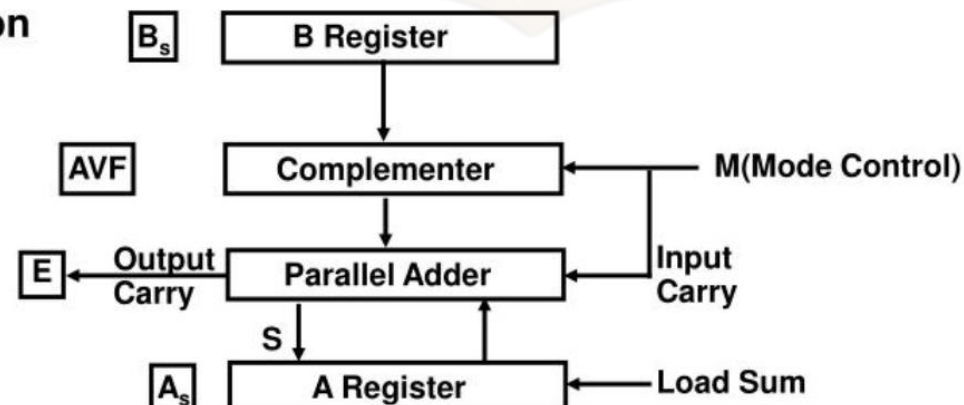
SIGNED MAGNITUDE ADDITION AND SUBTRACTION

Addition: $A + B$; A: Augend; B: Addend

Subtraction: $A - B$: A: Minuend; B: Subtrahend

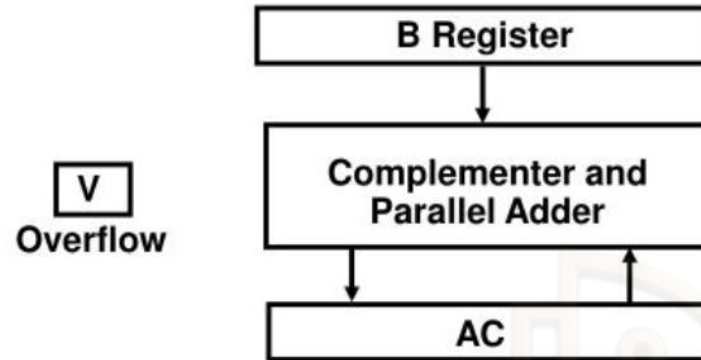
Operation	Add Magnitude	Subtract Magnitude		
		When $A > B$	When $A < B$	When $A = B$
$(+A) + (+B)$	$+(A + B)$			
$(+A) + (-B)$		$+(A - B)$	$-(B - A)$	$+(A - B)$
$(-A) + (+B)$		$-(A - B)$	$+(B - A)$	$+(A - B)$
$(-A) + (-B)$	$-(A + B)$			
$(+A) - (+B)$		$+(A - B)$	$-(B - A)$	$+(A - B)$
$(+A) - (-B)$	$+(A + B)$			
$(-A) - (+B)$	$-(A + B)$			
$(-A) - (-B)$		$-(A - B)$	$+(B - A)$	$+(A - B)$

Hardware Implementation

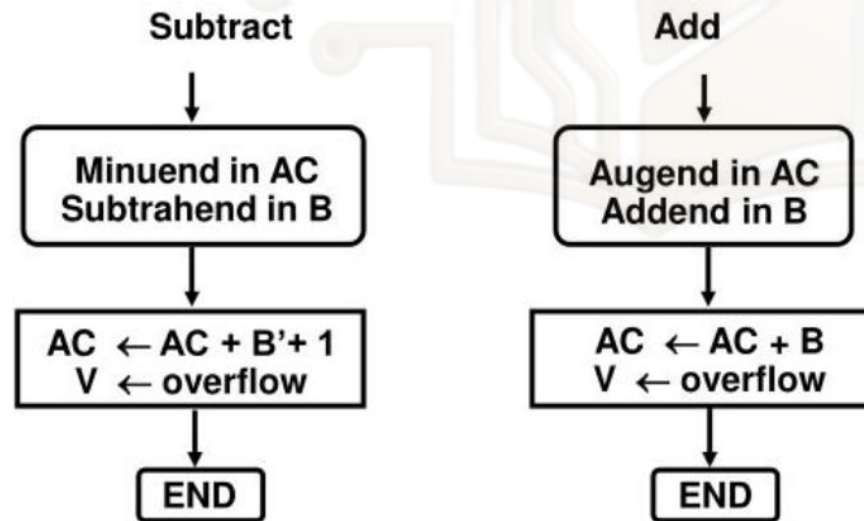


SIGNED 2'S COMPLEMENT ADDITION AND SUBTRACTION

Hardware



Algorithm



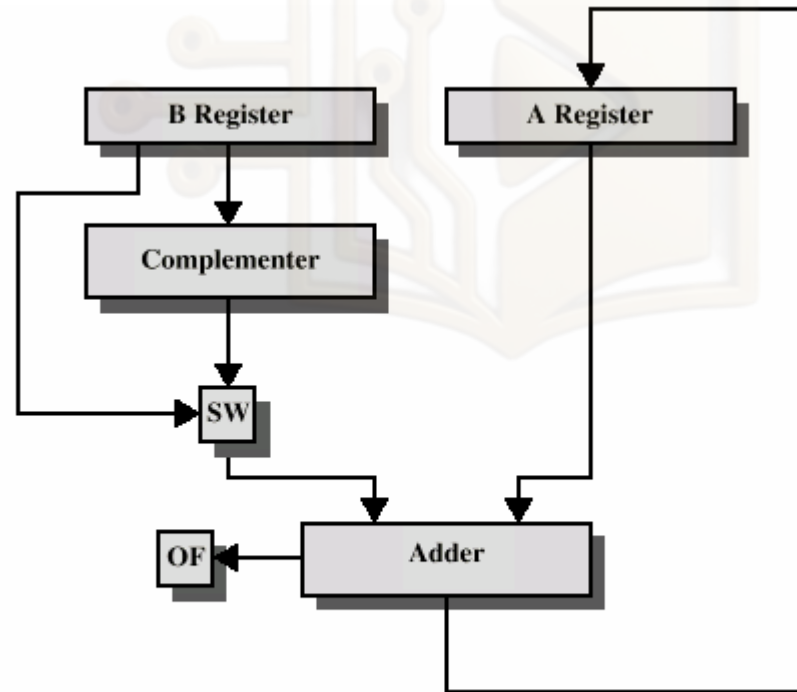
Unsigned Binary Addition and Subtraction

Addition and Subtraction:

Addition and Subtraction is done using following steps:

- Normal binary addition
- Monitor sign bit for overflow
- Take two's complement of subtrahend and add to minuend ,i.e. $a - b = a + (-b)$

Hardware for Addition and Subtraction:



OF = overflow bit

SW = Switch (select addition or subtraction)

OVERFLOW RULE: If two numbers are added, and they are both positive or both negative, then overflow occurs if and only if the result has the opposite sign.

$\begin{array}{r} 1001 = -7 \\ + 0101 = 5 \\ \hline 1110 = -2 \end{array}$ (a) $(-7) + (+5)$	$\begin{array}{r} 1100 = -4 \\ + 0100 = 4 \\ \hline 10000 = 0 \end{array}$ (b) $(-4) + (+4)$
$\begin{array}{r} 0011 = 3 \\ + 0100 = 4 \\ \hline 0111 = 7 \end{array}$ (c) $(+3) + (+4)$	$\begin{array}{r} 1100 = -4 \\ + 1111 = -1 \\ \hline 11011 = -5 \end{array}$ (d) $(-4) + (-1)$
$\begin{array}{r} 0101 = 5 \\ + 0100 = 4 \\ \hline 1001 = \text{Overflow} \end{array}$ (e) $(+5) + (+4)$	$\begin{array}{r} 1001 = -7 \\ + 1010 = -6 \\ \hline 10011 = \text{Overflow} \end{array}$ (f) $(-7) + (-6)$

Figure 9.3 Addition of Numbers in Twos Complement Representation

SUBTRACTION RULE: To subtract one number (subtrahend) from another (minuend), take the two's complement (negation) of the subtrahend and add it to the minuend.

Thus, subtraction is achieved using addition, as illustrated in Figure 9.4. The last two examples demonstrate that the overflow rule still applies.

$\begin{array}{r} 0010 = 2 \\ + 1001 = -7 \\ \hline 1011 = -5 \end{array}$ (a) $M = 2 = 0010$ $S = 7 = 0111$ $-S = 1001$	$\begin{array}{r} 0101 = 5 \\ + 1110 = -2 \\ \hline 10011 = 3 \end{array}$ (b) $M = 5 = 0101$ $S = 2 = 0010$ $-S = 1110$
$\begin{array}{r} 1011 = -5 \\ + 1110 = -2 \\ \hline 11001 = -7 \end{array}$ (c) $M = -5 = 1011$ $S = 2 = 0010$ $-S = 1110$	$\begin{array}{r} 0101 = 5 \\ + 0010 = 2 \\ \hline 0111 = 7 \end{array}$ (d) $M = 5 = 0101$ $S = -2 = 1110$ $-S = 0010$
$\begin{array}{r} 0111 = 7 \\ + 0111 = 7 \\ \hline 1110 = \text{Overflow} \end{array}$ (e) $M = 7 = 0111$ $S = -7 = 1001$ $-S = 0111$	$\begin{array}{r} 1010 = -6 \\ + 1100 = -4 \\ \hline 10110 = \text{Overflow} \end{array}$ (f) $M = -6 = 1010$ $S = 4 = 0100$ $-S = 1100$

Figure 9.4 Subtraction of Numbers in Twos Complement Representation ($M - S$)

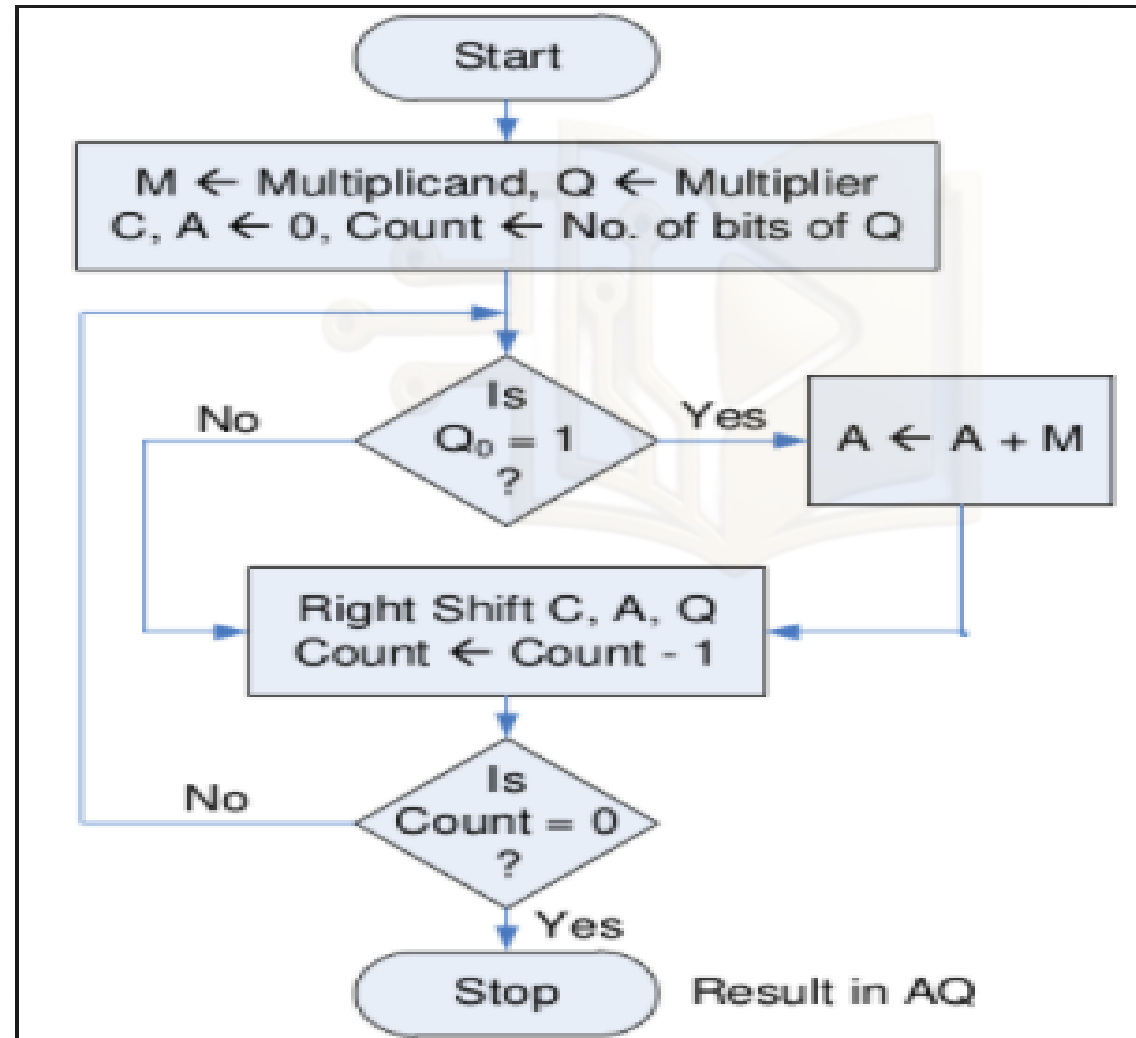
Multiplication Example (unsigned) (long hand)

- 1011 Multiplicand (11 dec)
- x 1101 Multiplier (13 dec)
- 1011 Partial products
- 00000 Note: if multiplier bit is 1 copy
- 101100 multiplicand (place value)
- 1011000 otherwise zero
- 10001111 Product (143 dec)
- Note: need double length result

Shift-add multiplication:

=> multiplication of unsigned numbers.

=> multiplication can be performed by repeated addition of multiplicand and shifting



Multiply [1011(M) and 1101(Q)]

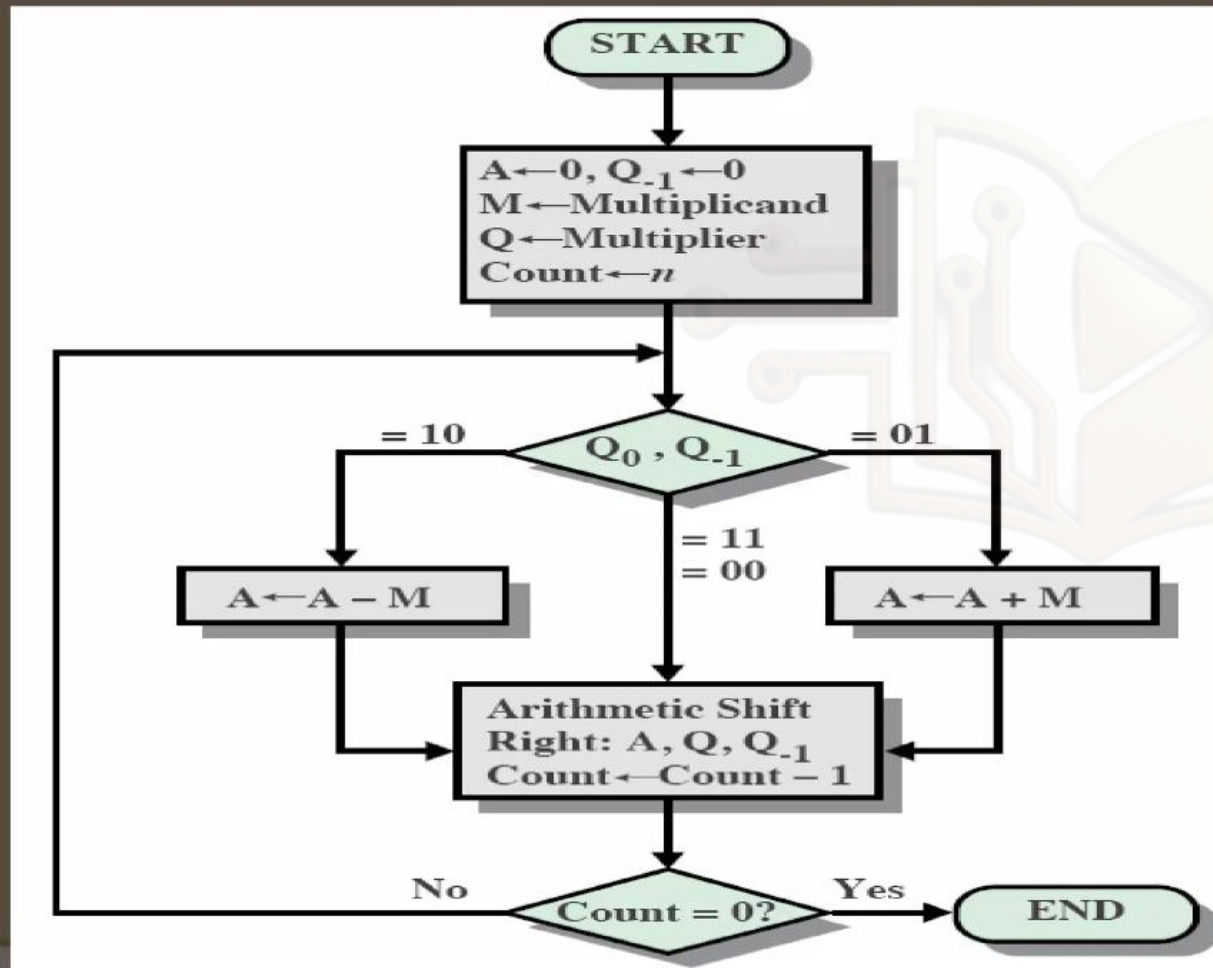
C	A	Q	M	Initial Values		
0	0000	1101	1011			
0	1011	1101	1011	Add	}	First Cycle
0	0101	1110	1011	Shift		
0	0010	1111	1011	Shift	}	Second Cycle
0	1101	1111	1011	Add		
0	0110	1111	1011	Shift	}	Third Cycle
1	0001	1111	1011	Add		
0	1000	1111	1011	Shift	}	Fourth Cycle

C	A	Q	M	operations
0	0000	1101	1011	Initial values
0	1011	1101	1011	Add(A+M)
0	0101	1110	1011	Shift(right)
0	0010	1111	1011	Shift(right)
0	1101	1111	1011	Add(A+M)
0	0110	1111	1011	Shift(right)
1	0001	1111	1011	Add
0	1000	1111	1011	Shift(right)

Answer is: AQ = $(10001111)_2 = 13 \times 11 = 143$

Booth's Algorithm

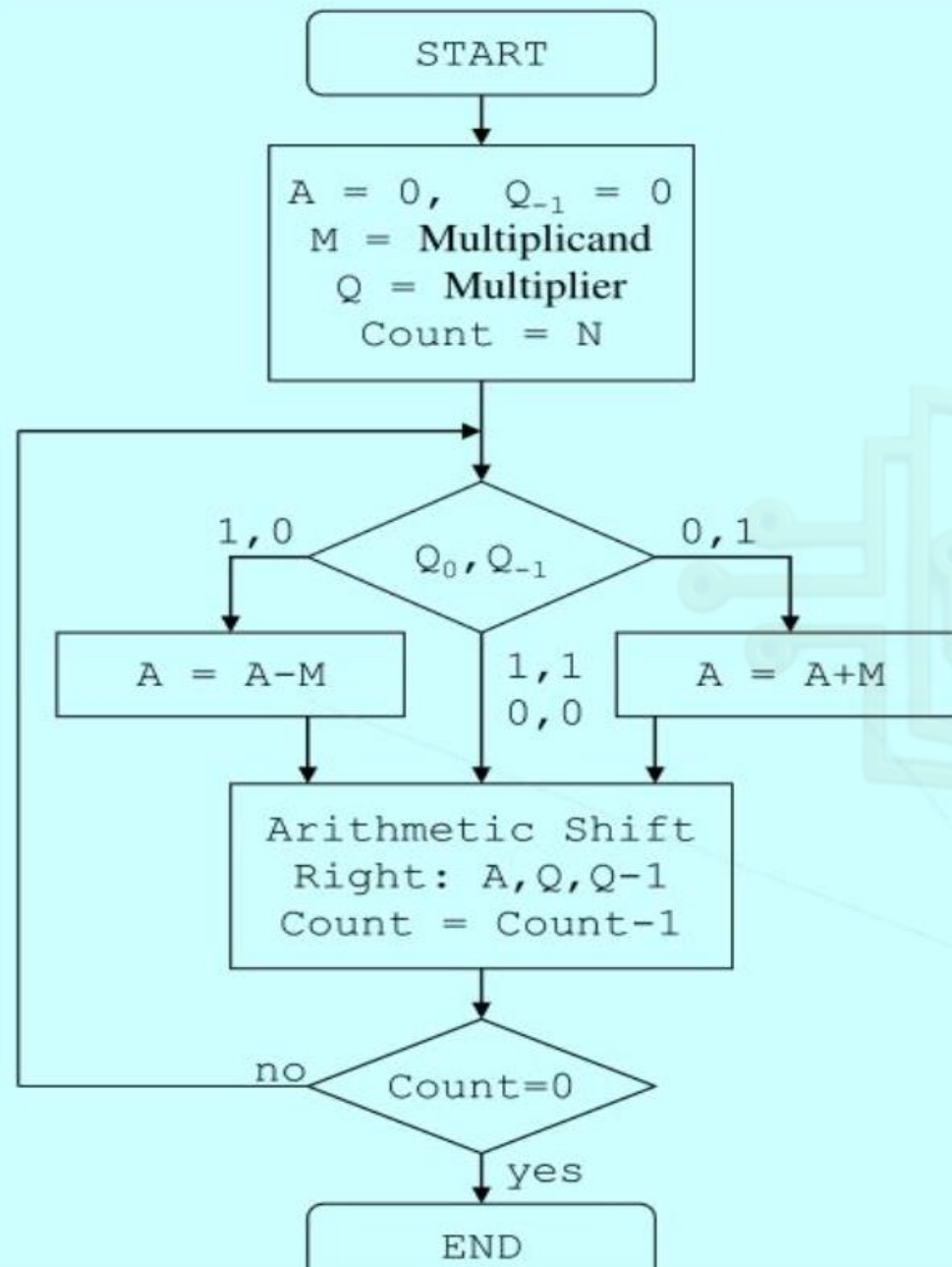
Flow chart



Booth : (7) x (-3)

A	Q	Q-1 (-3)	M	7	
0000	1101	0	0111		
1001	1101	0	0111	A <- (A - M)	1st cycle
1100	1110	1	0111	Shift	
0011	1110	1	0111	A <- (A + M)	2nd cycle
0001	1111	0	0111	Shift	
1010	1111	0	0111	A <- (A - M)	3rd cycle
1101	0111	1	0111	Shift	
1110	1011	1	0111	Shift	

Booth's Algorithm for 2's Complement Multiplication



$M = 0101$ $-M = 1011$
 $Q = 0110$
 $N = 4$

A	Q	Q_{-1}	N
0000	0110	0	4
0000	0011	0	3
1011	0011	0	
1101	1001	1	2
1110	1100	1	1
0011	1100	1	
0001	1110	0	0

Answer is in A, Q

$00011110_2 = 30_{10}$

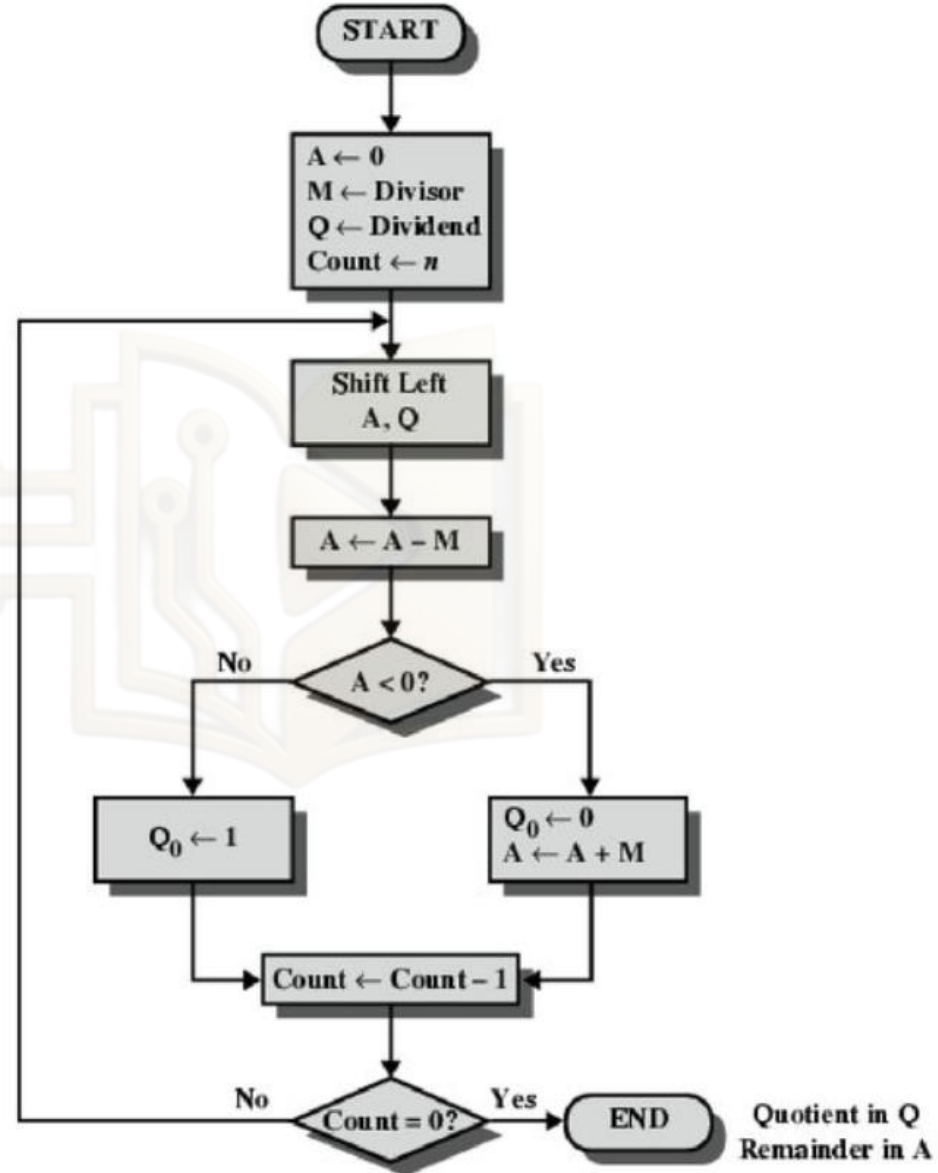
Two's complement multiplication can be performed using Booth's Algorithm.

To save time, both M and $-M$ can be computed ahead and provided as one of the inputs to the adders for $A-M$ and $A+M$.

Instead of a loop, the math processor can provide separate hardware for each stage of

Unsigned Binary Division

- Flowchart



Unsigned Binary Division

- Example using division of the unsigned integer 7 by the unsigned integer 3

A	Q	M = 0011	
0000	0111	Initial values	
0000	1110	Shift	1
1101		A = A - M	
0000	1110	A = A + M	
0001	1100	Shift	2
1110		A = A - M	
0001	1100	A = A + M	
0011	1000	Shift	3
0000		A = A - M	
0000	1001	$Q_0 = 1$	
0001	0010	Shift	4
1110		A = A - M	
0001	0010	A = A + M	

1. Algorithm & Example-1

Restoring Division Algorithm For Unsigned Integer

Algorithm

Restoring Division Algorithm For Unsigned Integer Steps	
Step-1:	Initialize registers Q = Dividend, M = Divisor, $A = 0$, n = number of bits in dividend
Step-2:	Content of register A and Q is shifted left as by considering them as a single unit AQ
Step-3:	Content of register M is subtracted from A and result is stored in A (i.e. $A = A - M$)
Step-4:	The MSB (most significant bit) of the A is checked if it is 0 then LSB (least significant bit) of Q is set to 1 otherwise if it is 1 then LSB of Q is set to 0 and value of register A is restored.
Step-5:	The value of counter n is decremented by 1
Step-6:	If value of n becomes 0 then goto next step otherwise repeat from step-2
Step-7:	Now register Q contains Quotient value and A contains Remainder value

Example-1

Find 11 divided by 3 using restoring division Algorithm method

Solution:

Dividend = 11

Divisor = 3

First the registers are initialized with corresponding values (Q = Dividend, M = Divisor, A = 0, n = number of bits in dividend)

n	M	A	Q	Operation
4	00011	00000	1011	initialize
4	00011	00001	011_	shift left AQ
	00011	11110	011_	A=A-M
	00011	00001	0110	Q[0]=0 And restore A
3	00011	00010	110_	shift left AQ
	00011	11111	110_	A=A-M
	00011	00010	1100	Q[0]=0 And restore A
2	00011	00101	100_	shift left AQ
	00011	00010	100_	A=A-M
	00011	00010	1001	Q[0]=1
1	00011	00101	001_	shift left AQ
	00011	00010	001_	A=A-M
	00011	00010	0011	Q[0]=1

register Q contain the quotient 3 and register A contain remainder 2

BCD Arithmetic Unit (BCD Adder)

- **Procedure:**
 - **Step 1:** Add the two BCD numbers, using the rules for binary addition
 - **Step 2:** If a 4-bit sum is equal to or less than 9, it is a valid BCD number.
 - **Step 3:** If a 4-bit sum is greater than 9, or if a carry out of the 4-bit group is generated, it is an invalid result. Add 6 (0110) to the 4-bit sum in order to skip the six invalid states and return the code to 8421. If a carry results when 6 is added, simply add the carry to the next 4-bit group.

When you add BCD think decimal and answer in Binary

478
+137

1
0100
0001

1
0111
0011

1000
0111

0110
0000

1011
0110

1111
0110

615

0110

0001

0101

- A decimal adder requires a minimum of nine inputs and five outputs, since four bits are required to code each decimal digit and the circuit must have an input and output carry.
- Consider the arithmetic addition of two decimal digits in standard BCD code (8421 code), together with an input carry from a previous stage.
 - Since each input digit does not exceed 9, the output sum cannot be greater than $9+9+1=19$, the 1 in the sum being an input carry.
 - Apply 2 BCD digits to a 4-bit binary adder.
 - The adder will form the sum in binary and produce a result that ranges from 0 through 19.
 - These binary numbers are listed in the table and are labeled by K, Z_8, Z_4, Z_2, Z_1 and K is carry.
 - The columns under the binary sum list the binary value that appears in the outputs of the 4-bit binary adder

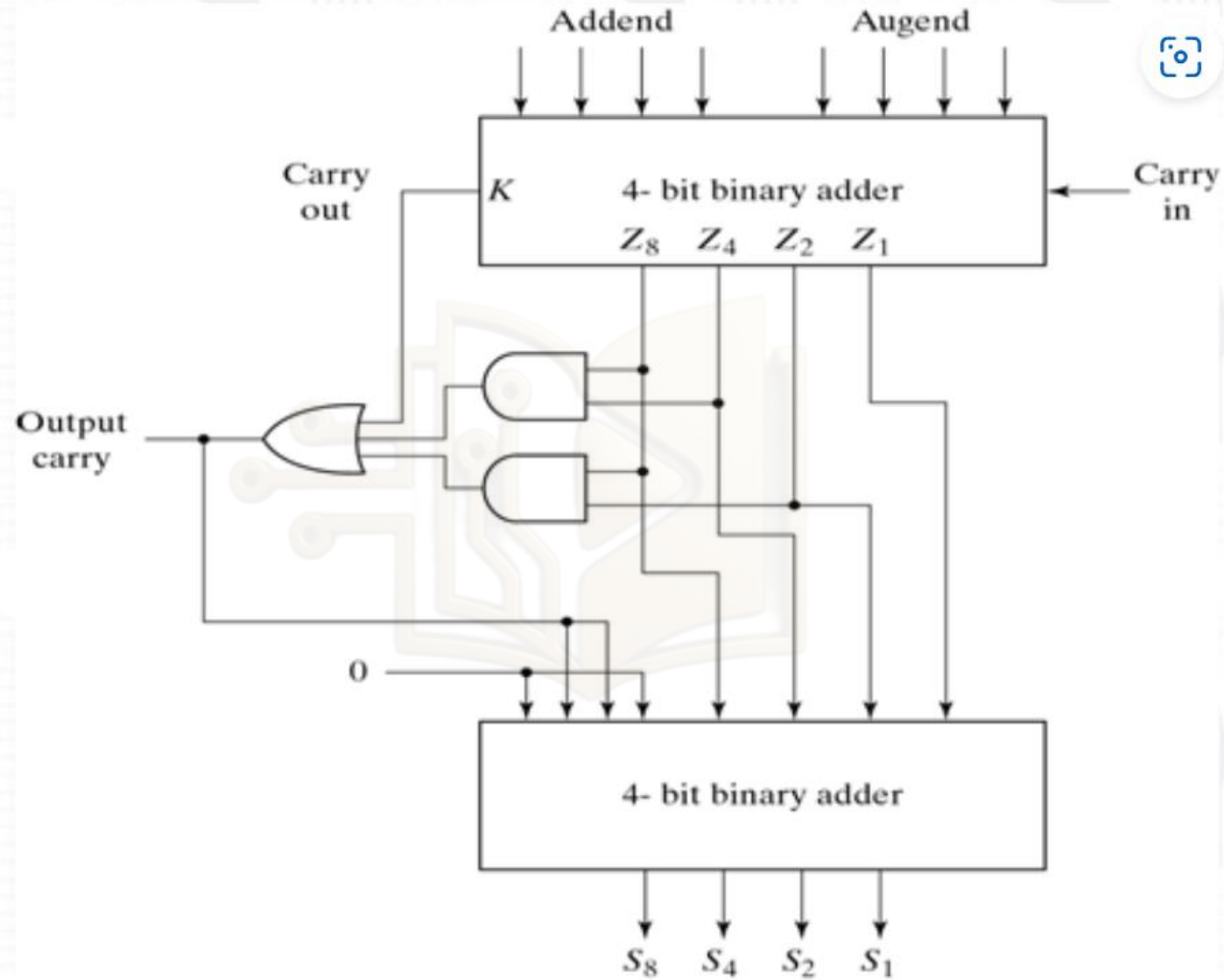
Decimal Adder for standard BCD CODE

K	Z ₈	Z ₄	Z ₂	Z ₁	C	S ₈	S ₄	S ₂	S ₁	D	K	Z ₈	Z ₄	Z ₂	Z ₁	C	S ₈	S ₄	S ₂	S ₁	D
0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	1	0	0	0	0	10
0	0	0	0	1	0	0	0	0	1	1	0	1	0	1	1	1	0	0	0	1	11
0	0	0	1	0	0	0	0	1	0	2	0	1	1	0	0	1	0	0	1	0	12
0	0	0	1	1	0	0	0	1	1	3	0	1	1	0	1	1	0	0	1	1	13
0	0	1	0	0	0	0	1	0	0	4	0	1	1	1	0	1	0	1	0	0	14
0	0	1	0	1	0	0	1	0	1	5	0	1	1	1	1	1	0	1	0	1	15
0	0	1	1	0	0	0	1	1	0	6	1	0	0	0	0	1	0	1	1	0	16
0	0	1	1	1	0	0	1	1	1	7	1	0	0	0	1	1	0	1	1	1	17
0	1	0	0	0	0	1	0	0	0	8	1	0	0	1	0	1	1	0	0	0	18
0	1	0	0	1	0	1	0	0	1	9	1	0	0	1	1	1	1	0	0	1	19

Boolean function for correction

- From the table, when the binary sum is equal to or less than 1001, the corresponding BCD number is identical, and therefore no conversion is needed.
- When the binary sum is greater than 1001, non valid BCD representation is obtained.
- Addition of binary 6 (0110) to the binary sum converts it to the correct BCD representation and also produces an output carry as required
- A correction is needed when the binary sum has an output carry $K=1$. The other six combinations from 1010 through 1111 that need a correction have a 1 in position Z_8 . To distinguish them from binary 1000 and 1001, which also have a 1 in position Z_8 , it can be concluded that Z_4 or Z_2 must have a 1.
- Therefore, the condition for a correction and an output carry can be expressed by the Boolean function:
$$C=K+Z_8 Z_4+Z_8 Z_2$$
- When $C = 1$, it is necessary to add 0110 to the binary sum and provide an output carry for the next stage.

Block Diagram of a BCD Adder



- The two decimal digits, together with the input carry, are first added in the top 4-bit adder to produce the binary sum.
- When the output carry is equal to zero, nothing is added to the binary sum
- When it is equal to 1, binary 0110 is added to the binary sum through the bottom 4-bit adder.
- A decimal parallel adder that adds n decimal digits needs n BCD adder stages.
- The output carry from one stage must be connected to the input carry of the next higher-order stage.

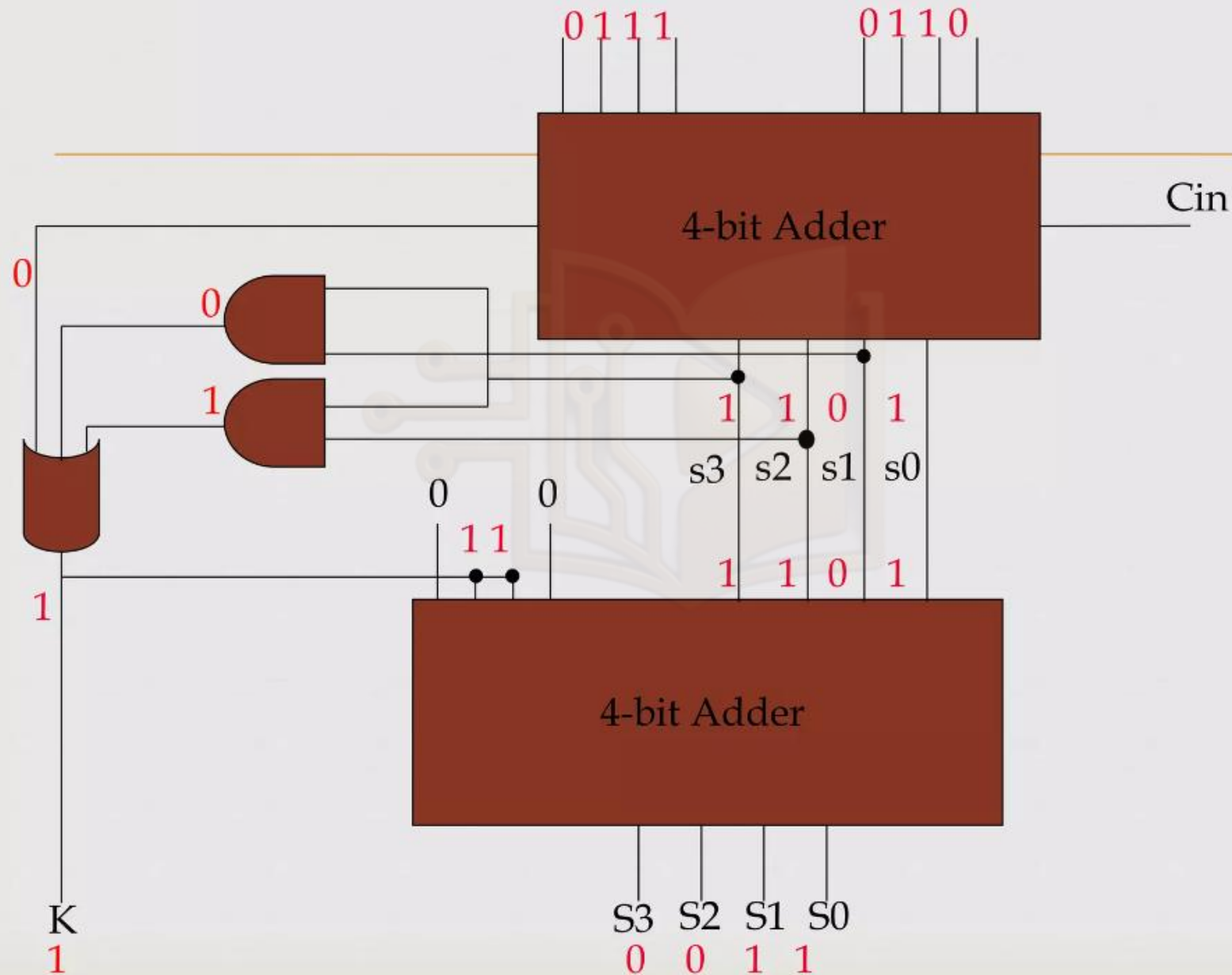
Why add 6 to the result?



- Sum of two BCD digits exceeding 9.
- A carry is generated.
- Converting the invalid digit into valid digit.
- Carry generated by adding 6 to the invalid BCD digit's passed on to the next BCD digit.

$$\begin{array}{r} 0101 \\ + 0110 \\ \hline 1011 \rightarrow \text{Invalid BCD number} \\ + 0110 \rightarrow \text{Add 6} \\ \hline 0001\ 0001 \rightarrow \text{Valid BCD number} \end{array}$$

4-bit bcd adder

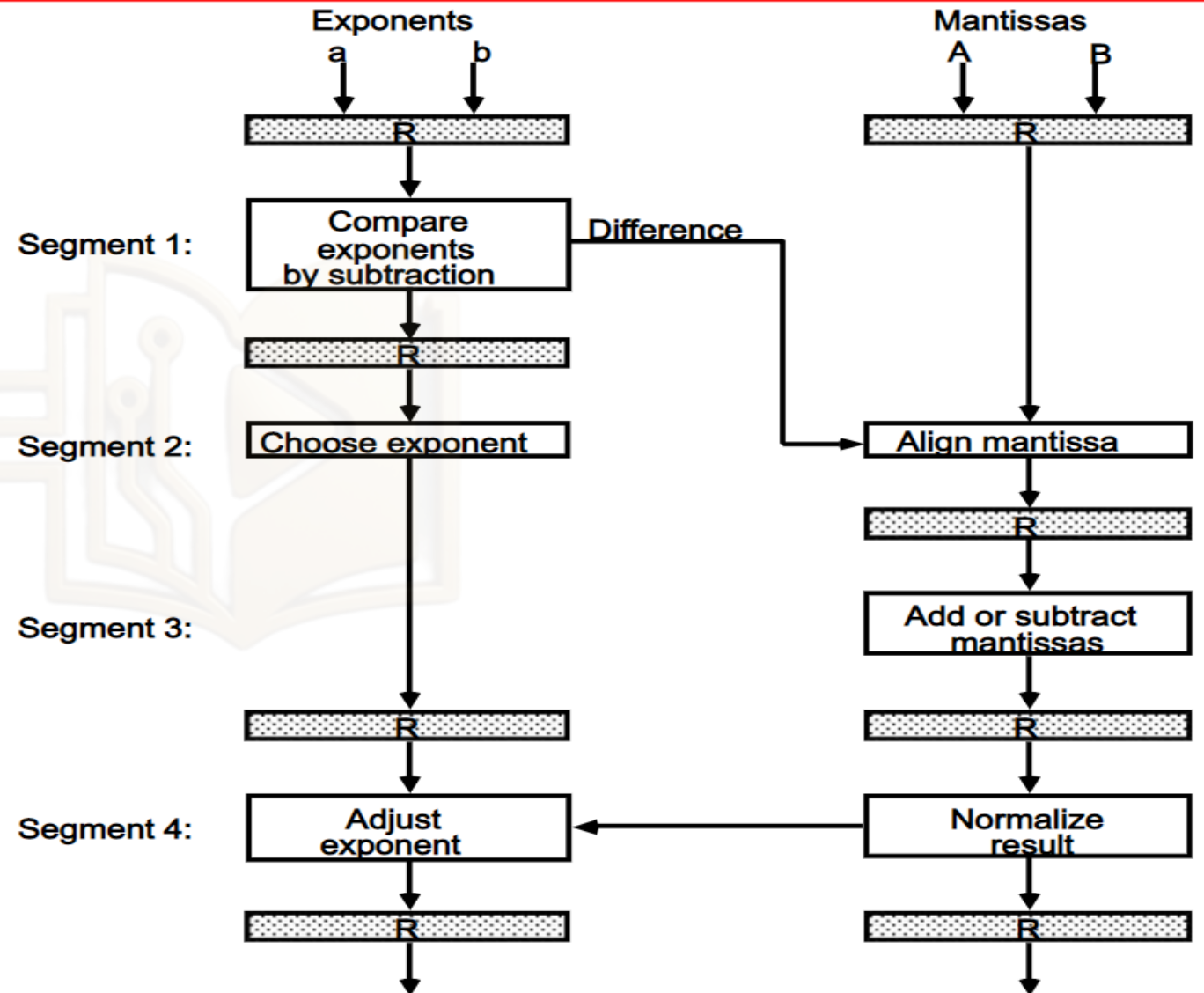


Arithmetic Pipelining

Floating-point adder

$$X = A \times 2^a$$
$$Y = B \times 2^b$$

- [1] Compare the exponents
- [2] Align the mantissa
- [3] Add/sub the mantissa
- [4] Normalize the result



Floating point addition using arithmetic pipeline :

The following sub operations are performed in this case:

1. Compare the exponents.
2. Align the mantissas.
3. Add or subtract the mantissas.
4. Normalize the result

First of all the two exponents are compared and the larger of two exponents is chosen as the result exponent. The difference in the exponents then decides how many times we must shift the smaller exponent to the right. Then after shifting of exponent, both the mantissas get aligned. Finally the addition of both numbers take place followed by normalisation of the result in the last segment.

Example:

Let us consider two numbers,

$$X=0.3214*10^3 \text{ and } Y=0.4500*10^2$$

Explanation:

First of all the two exponents are subtracted to give $3-2=1$. Thus 3 becomes the exponent of result and the smaller exponent is shifted 1 times to the right to give

$$Y=0.0450*10^3$$

Finally the two numbers are added to produce

$$Z=0.3664*10^3$$

As the result is already normalized the result remains the same.

